seaPilot — Project Write-up

Team seaLion · Sea x OpenAI Regional Codex Hackathon Vietnam · 31 Oct 2026

Build direction: 02 — AI-Native Products & Operations

- Repo: https://github.com/Phonghigh/seaLion-codex-hackthon-sea-openai
- Live demo, customer: https://seapilot-customer-hoang.wirybear257.chatgpt.site
- Live demo, customer service: https://seapilot-service-hoang.wirybear257.chatgpt.site/?demo=1

1. Problem

In e-commerce customer service, one complaint often takes several contacts. A customer reports a missing item in chat. The next day they write again in a new conversation, and a different agent picks it up. That agent starts from zero. They either re-read old threads or ask the customer to explain again and resend the same photos.

Meanwhile the facts change: a parcel that was "in transit" yesterday is "delivered" today. The result is duplicate tickets, repeated questions and wrong next steps.

2. Solution

seaPilot is a case workspace where the **case, not the conversation, is the unit of work**.

- **Triage decides who answers.** Each new message is classified by category, confidence and risk.
  - Simple, low-risk questions that our data can answer are answered by AI on its own: order and parcel status, basic order details (order code, date, parcels, items) and receipt confirmations.
  - Everything else goes to a human. The rules are in a config file, not in the prompt, and routing considers the agent's team, shift and language.
- **The right human gets the full case.** When a customer writes again, seaPilot:
  - suggests the existing case and says why it matches; the agent confirms the link in one click;
  - shows the full history and evidence of that case on one screen;
  - compares current order and shipment data with what the customer was last told, and lists what changed;
  - names the one fact still missing, and drafts a reply that asks only that;
  - proposes the next action, such as an escalation to operations, with the evidence attached.
- **Humans approve.** Every fact is labelled by source: customer statement, system data, or unverified. An agent approves every reply and every action. A reply says "forwarded" only after the action has succeeded, and every result is logged.

3. Demo story (runs live on the public Sites)

- **Scene 0:** Minh asks "Where is my order DH002?". Triage marks it low-risk and answerable from shipment data, so the AI replies by itself.
- **Scene 1:** Lan ordered a T-shirt and a cap (order DH001, two parcels) and reports the cap missing, with a photo. A photo plus a missing item means a human decides, so triage routes it to An (delivery team, on shift). The suggestion shows that parcel B is still in transit. The agent approves, and case CS001 is created.
- **Scene 2:** Next day, the carrier marks parcel B as delivered and An is off shift. Lan opens a new conversation, annoyed. Binh, a different agent, links it to CS001 in one click and sees the earlier

photo and the status change. The draft does not ask for the photo again. It asks one question: did parcel B not arrive, or did it arrive without the cap?

- **Scene 3:** Lan says the parcel arrived without the cap and sends a second photo. seaPilot proposes an escalation to operations with both photos. Binh approves, the task is created, and the case moves to "waiting for operations".
- **Scene 4:** A vague message with low triage confidence goes to a human: Carlos, General Support, on shift, English-speaking.

4. Why AI-native

Without AI, keeping one case consistent across conversations, agents and shifts needs a person to re-read everything at every handover. In seaPilot the case itself remembers, notices what changed and asks the next right question. AI handles the easy questions and leaves the risky ones to people.

What a business can now do: let any agent on any shift pick up any case without losing context, and stop asking customers to repeat themselves.

5. How it works

- **Apps:** a pnpm monorepo with two Next.js apps (customer chat and CS workspace), a Hono API in TypeScript, and shared Zod contracts.
- **Data:** PostgreSQL 16 with Drizzle stores cases, conversations, messages, evidence, events and tasks. Context lives in the database, not in the model's memory.
- **AI:** OpenAI Responses API with:
  - a structured triage call (category, confidence, risk), followed by deterministic routing rules;
  - one agent with three read-only function tools: `find_cases`, `get_case_context`, `get_order_shipments`.
    The backend validates every tool call and every model output with Zod. The model never writes to the database.
- **Approval:** an approval API executes replies and actions only after a human approves, and logs every result.
- **Hosting:** both web apps are static exports published on ChatGPT Sites. For the demo, the API and database run on a team laptop behind a Cloudflare tunnel.

6. How we used Codex

- **Preparation:** before coding we wrote `AGENTS.md`, the architecture and the task prompts (in `prompts/`) as plain text. Codex then built the product from them on the day.
- **What Codex built:**
  - the API, database schema, migration and seed;
  - the three tools and the agent loop;
  - triage and routing;
  - both web apps and their integration with the API;
  - the approval flow and the demo controls;
  - the tests;
  - publishing to ChatGPT Sites through the Sites plugin.
    Each task is logged with its start and end time in `docs/codex-log.md` (35 entries).

- **Usage evidence:** shared Codex hooks recorded 1,120 events across 17 sessions by 16:30, including 533 tool invocations. The report is in `docs/codex-usage-report.md`.
- **Tests:** 124 automated tests plus 30 tests for the usage-reporting tool. They include an end-to-end continuity test, written by Codex, that replays the demo story and checks for one case only, no repeated question, and the correct final status.
- **A concrete example:** in task FIX01, Codex reproduced a test that failed about 1 run in 5. It traced the failure to database state shared between test files and fixed it by isolating test data, without adding retries.

7. Results and limits

- **Working today:** Scenes 0–4 run live end-to-end with the real model:
  - triage and human-or-AI routing;
  - case linking across conversations;
  - change detection;
  - one-question drafts;
  - an approved escalation with evidence.
- **Simulated:** the order, shipment and operations systems; a single channel (chat); a demo clock for "next day".
- **Next:** real carrier and order APIs, email and app channels, and a pilot to measure case-link accuracy and repeated questions avoided.